

Introduction to *DART* Programming Language

พื้นฐานที่ควรทราบก่อนใช้งาน Flutter SDK

By Prasertsak U., Ph.D

Dart Language

- Dart is an object-oriented programming (OOP) language, has a class-based style.
- Uses a C-style syntax. If you are familiar with the C#, C++, Swift, Kotlin, and Java/JavaScript languages, you will be able to start developing in Dart quickly

Basic Comments

- Single-line comments begin with //

```
void main() {  
    // TODO: refactor into an AbstractLlamaGreetingFactory?  
    print('Welcome to my Llama farm!');  
}
```

- Multiline comments begin with /* and end with */

```
void main() {  
    /*  
     * This is a lot of work. Consider raising chickens.  
     */  
  
    Llama larry = Llama();  
    larry.feed();  
    larry.exercise();  
    larry.clean();  
    */  
}
```

Starting with Hello World

- เปิดโปรแกรม Text Editor (ใช้ VS-Code) เลือก New Text File และเขียนตัวอย่างโปรแกรมต่อไปนี้

```
void main() {  
    print("Hello world using Dart Language!");  
}
```

- บันทึกไฟล์โดยใช้ชื่อไฟล์ hello.dart
- เปิด terminal บน VS-Code และไปยัง folder ที่บันทึกไฟล์ จากนั้นใช้คำสั่งรันโปรแกรม ดังนี้



The screenshot shows the VS Code interface with the 'TERMINAL' tab selected. The terminal output shows the following commands and results:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\gnili> cd .\Downloads  
PS C:\Users\gnili\Downloads> dart hello.dart  
Hello, World!  
PS C:\Users\gnili\Downloads> 
```

Variables

- ตัวแปรพื้นฐานที่ควรทราบ

`int` - เก็บตัวเลขจำนวนเต็ม

`double` - เก็บตัวเลขทศนิยม

`num` - เก็บได้ทั้งจำนวนเต็มและทศนิยม

`bool` - เก็บค่า `true/false`

`String` - เก็บข้อมูลชนิดข้อความ (ไม่สามารถนำไปคำนวณได้)

`List` - เก็บข้อมูลในลักษณะ `array`

`Map` - เก็บข้อมูลในรูปแบบของ `key: value`

`Set` - เก็บข้อมูลในลักษณะกลุ่มข้อมูลที่เน้นความไม่ซ้ำกันของสมาชิก และไม่สนใจลำดับ

การอ้างถึงค่าที่เก็บในตัวแปร

- ใช้เครื่องหมาย \$ นำหน้าชื่อตัวแปรที่ต้องการนำค่าที่เก็บมาใช้ ตัวอย่าง เช่น

```
1 void main() {  
2     num x = 10;  
3     num y = 20.5;  
4     print("The sum of $x and $y is ${x + y}"); // The sum of 10 and 20.5 is 30.5  
5  
6 }
```

ประกาศด้วย num เก็บค่าได้ทั้งจำนวนเต็ม และจำนวนทศนิยม

หรือ...

```
1 void main() {  
2     int x = 10;  
3     double y = 20.5;  
4     print("The sum of $x and $y is ${x + y}"); // The sum of 10 and 20.5 is 30.5  
5  
6 }
```

แยกประกาศด้วย int และ double เพื่อเก็บค่าจำนวนเต็ม และทศนิยม

การใช้เครื่องหมาย \$ อ้างถึงค่าในตัวแปร

- การใช้ ***\$variableName*** (แบบไม่มีปีกกา): เราจะใช้รูปแบบนี้เมื่อต้องการอ้างถึง "ชื่อตัวแปรเดี่ยวๆ" เท่านั้น โดยที่ไม่มีการเรียก Property หรือ Method ต่อท้าย
 - เหมาะสำหรับ: ตัวแปรพื้นฐาน เช่น String, int, double หรือ bool
 - ข้อจำกัด: ไม่สามารถใช้จุด . เพื่อเข้าถึง Property หรือเรียกฟังก์ชันได้
- การใช้ ***\${expression}*** (แบบมีปีกกา): เราจะใช้รูปแบบนี้เมื่อสิ่งที่ต้องการแสดง "มีความซับซ้อนมากกว่าแค่ชื่อตัวแปร" หรือเป็นนิพจน์ (Expression)
 - การเข้าถึง Property หรือ Method: เช่น \${user.name} หรือ \${items.length}
 - การคำนวณทางคณิตศาสตร์: เช่น \${a + b}
 - การเรียกใช้ Function: เช่น \${getName()}
 - การใช้ Ternary Operator: เช่น \${score >= 50 ? 'Pass' : 'Fail'}
 - กรณีที่ตัวแปรติดกับตัวอักษรอื่น: เพื่อป้องกันความสับสนของ Compiler เช่น \${unit}meters

การใช้เครื่องหมาย \$ อ้างอิงค่าในตัวแปร (cont.)

- การเข้าถึง Property ของ Object หากต้องการเข้าถึง Property ของ Object ตรงๆ เช่น ตัวแปร user ที่มีฟิลด์ name ห้ามใช้ `$user.name` เป็นอันตราย เพราะ Dart จะมองเห็นแค่ `$user` แล้วต่อด้วยข้อความ `.name` ต้องใช้ครอบปีกกาเสมอ เมื่อมีการใช้เครื่องหมายจุด . เช่น `"${user.name}"`

```
class User {  
  String name;  
  int age;  
  // Constructor สำหรับกำหนดค่าเริ่มต้น  
  User(this.name, this.age);  
}
```

```
void main() {  
  // 2. สร้าง Object หรือประกาศตัวแปร user  
  User user = User('สมศักดิ์', 25);  
  // 3. การแสดงผลโดยใช้ String Interpolation  
  // แบบที่ผิด: หากไม่ใส่ปีกกา Dart จะอ่านค่าแค่ $user แล้วแสดงข้อความ .name ต่อท้ายตรงๆ  
  print("สวัสดีคุณ $user.name อายุของคุณคือ $user.age ปี");  
  // ผลลัพธ์ที่ผิดพลาดจะได้ประมาณ: สวัสดีคุณ Instance of 'User'.name  
  // แบบที่ถูกต้อง: ต้องใช้ปีกกาครอบเมื่อมีการเข้าถึง Property หรือ Method ต่อท้าย  
  print("สวัสดีคุณ ${user.name} อายุของคุณคือ ${user.age} ปี"); // ผลลัพธ์: สวัสดีคุณ สมศักดิ์ อายุของคุณคือ 25 ปี  
}
```


คำว่า var กับ dynamic

- เป็นการประกาศตัวแปรโดยไม่ต้องระบุชนิด
- ชนิดของตัวแปรจะถูกกำหนดในขณะที่มีการใส่ค่าเข้าไปให้กับตัวแปรนั้น ซึ่งตัวแปรนั้น ๆ จะมีชนิดใดจะขึ้นอยู่กับค่าที่เก็บลงไป
- เมื่อประกาศด้วย **var** ชนิดของตัวแปรจะถูกกำหนดในครั้งแรกของการใส่ค่า และจะเป็นชนิดนั้น ๆ ไปตลอดเปลี่ยนไม่ได้
- เมื่อประกาศด้วย **dynamic** ชนิดของตัวแปรจะถูกกำหนดใหม่ในทุก ๆ ครั้งของการใส่ค่า ชนิดจะเปลี่ยนแปลงไปได้เรื่อย ๆ

คำว่า var กับ dynamic (cont.)

- ตัวอย่าง

การใช้เครื่องหมายคร่อมค่าข้อความใด ๆ ด้วย “ ” หรือ ‘ ’
จะให้ผลเหมือนกันใช้แทนกันได้ เช่น “somsak” หรือ ‘somsak’ ก็ให้ผลเหมือนกัน

```
1 void main() {  
2   var x = 'Somsak';  
3   print('value of x = $x, type is ${x.runtimeType}'); // value of x = Somsak, type is String  
4 }
```

- เมื่อพยายามจะเปลี่ยนแปลงค่าตัวแปร x ให้มีค่าเท่ากับ 10 จะเกิด Error เพราะ 10 ไม่ใช่ String

```
1 void main() {  
2   var x = 'Somsak';  
3   print('value of x = $x, type is ${x.runtimeType}'); // value of x = Somsak, type is String  
4   x = 10;
```

❌ hello.dart 1 of 1 problem

A value of type 'int' can't be assigned to a variable of type 'String'.
Try changing the type of the variable, or casting the right-hand type to 'String'. dart([invalid_assignment](#))

```
5 }
```

คำว่า var กับ dynamic (cont.)

- ตัวอย่าง dynamic

```
1 void main() {  
2     dynamic x = 'Somsak';  
3     print('value of x = $x, type is ${x.runtimeType}'); // value of x = Somsak, type is String  
4 }
```

- เมื่อพยายามจะเปลี่ยนแปลงค่าตัวแปร x ให้มีค่าเท่ากับ 10 จะไม่เกิดปัญหา และชนิดของตัวแปร x จะถูกเปลี่ยนเป็น int

```
1 void main() {  
2     dynamic x = 'Somsak';  
3     print('value of x = $x, type is ${x.runtimeType}'); // value of x = Somsak, type is String  
4     x = 10;  
5     print('value of x = $x, type is ${x.runtimeType}'); // value of x = 10, type is int  
6 }
```

คำว่า var กับ dynamic (cont.)

- **ข้อสังเกต:** หากประกาศตัวแปรโดยใช้ var นำหน้า โดยไม่มีการระบุค่าเริ่มต้นตอนประกาศ ตัวแปรนั้นจะมีลักษณะเป็น dynamic ดังตัวอย่าง

```
1 void main() {  
2     var x;  
3  
4     x = 'Somsak';  
5     print('value of x = $x, type is ${x.runtimeType}'); // value of x = Somsak, type is String  
6  
7     x = 10.5;  
8     print('value of x = $x, type is ${x.runtimeType}'); // value of x = 10.5, type is double  
9 }
```

คำว่า const และ final

- เป็นการกำหนดค่าให้กับตัวแปรนั้น ๆ เพียงครั้งเดียว และจะ不会有การเปลี่ยนแปลงค่าของตัวแปรนั้นอีก (const: กำหนดตั้งแต่ช่วงการ compile แต่ final จะกำหนดค่าให้กับตัวแปรตอน runtime)
- รูปแบบ

```
// Without datatype
const variable_name = value;

// With datatype
const data_type variable_name = value
```

```
1 void main() {
2     const vat = 7.0;
3     const double discount = 0.1;
4     print("vat: $vat");
5     print("discount: $discount");
6 }
```

```
// Without datatype
final variable_name = value;

// With datatype
final data_type variable_name = value
```

```
1 void main() {
2     final vat = 7.0;
3     final double discount = 0.1;
4     print("vat: $vat");
5     print("discount: $discount");
6 }
```

ชนิดตัวแปร List

- เป็นชนิดตัวแปรที่เก็บข้อมูลได้หลายค่า เทียบได้กับตัวแปรชนิด `array`
- สามารถเพิ่มเติมจำนวนที่ต้องการจัดเก็บได้ตลอดเวลา
- การประกาศใช้ จะใช้คำว่า `List<ชนิดตัวแปร>` ชื่อตัวแปร = `[]`

```
1 void main() {  
2     List<double> score = [];  
3     score.add(50.5); // เพิ่มค่าใส่เข้าไปใน List  
4     score.add(65.0); // เพิ่มค่าใส่เข้าไปใน List ต่อจากตำแหน่งก่อนหน้า  
5     score.add(55);  
6     print(score); // [ 50.5, 65.0, 55.0 ]  
7 }
```

ชนิดตัวแปร List (cont.)

- List ที่เก็บข้อมูลได้หลายประเภทในตัวแปรเดียว

```
1 void main() {  
2  
3     List<dynamic> data = []; // List ที่สามารถเก็บข้อมูลได้หลายประเภท  
4     data.add(50); // เพิ่มค่าใส่เข้าไปใน List  
5     data.add("Hello"); // เพิ่มค่าใส่เข้าไปใน List ต่อจากตำแหน่งก่อนหน้า  
6     data.add(55.5);  
7     print(data); // [ 50, Hello, 55.5 ]  
8 }
```

- การประกาศตัวแปรชนิด List แบบ Fixed จำนวนที่ต้องการเก็บ

```
1 void main() {  
2  
3     List<dynamic> data = List.filled(3, 0, growable: false); // สร้าง List ที่ไม่สามารถเพิ่มขนาดได้  
4     data[0] = 50; // เพิ่มค่าใส่เข้าไปใน List  
5     data[1] = "Hello"; // เพิ่มค่าใส่เข้าไปใน List ต่อจากตำแหน่งก่อนหน้า  
6     data[2] = 55.5;  
7  
8     print(data); // [ 50, Hello, 55.5 ]  
9 }
```

List แบบ Fixed-Length จะใช้คำสั่ง add เพิ่มค่าไม่ได้

ชนิดตัวแปร List (cont.)

- ตัวอย่างการใช้งาน Methods ต่าง ๆ ของ List เช่น

```
// Adding items to a list
var ages = [21, 23, 34, 19, 45];

ages.add(42); // Adds 42 to the end of the list
ages.addAll([42, 37, 62]); // Adds 42, 37 and 62 to the end of the list
print(ages); // Output: [21, 23, 34, 19, 45, 42, 42, 37, 62]
```

```
var favouriteColors = ['red', 'red', 'orange', 'yellow', 'green'];

favouriteColors.remove('red'); // Removes the first 'red' in the list
favouriteColors.removeAt(1); // Removes the second item in the list
favouriteColors.removeLast(); // Removes the last item from the list
print(favouriteColors); // Output: [red, yellow]
```

- ยังมี Methods อื่นๆ อีกมากภายใน List ให้ศึกษาเพิ่มเติมด้วยตนเอง

ตัวแปร Map

- Map - An object that associates **keys** and **values**

```
1 void main() {
2     Map mapVariable = {
3         'id1': 'Toyota',
4         'id2': 'Honda',
5         'id3': 'Susuki',
6         'id4': 'BMW',
7     };
8     print(mapVariable);
9     // Change the value of third item with Key of id3
10    mapVariable['id3'] = 'Isuzu';
11    print('Get filter with id3: ${mapVariable['id3']}');
12 }
```

ตัวแปร Set

- **Set** เป็นกลุ่มข้อมูล (Collection) ประเภทหนึ่งในภาษา Dart ที่มีเอกลักษณ์เฉพาะตัว คือ "กลุ่มข้อมูลที่เน้นความไม่ซ้ำกันของสมาชิก และไม่สนใจลำดับ" เครื่องหมายที่ใช้คือ { }

คุณสมบัติ	List (รายการ)	Set (เซต)
ข้อมูลซ้ำกัน	ได้ (Duplicate allowed)	ไม่ได้ (Must be unique)
มีลำดับ (Index)	มี (เริ่มจาก [0], [1], [2]...)	ไม่มี (หาข้อมูลด้วยการระบุค่าโดยตรง)
เครื่องหมายที่ใช้	ใช้ Square brackets []	ใช้ Curly braces { }

ตัวแปร Set (cont.)

```
void main() {  
    // 1. การสร้าง Set ของสัตว์ (สังเกตว่ามี 'Cat' ซ้ำกัน 2 ตัว)  
    var animals = {'Cat', 'Dog', 'Bird', 'Cat'};  
  
    // ผลลัพธ์: Dart จะตัด 'Cat' ที่ซ้ำออกไปโดยอัตโนมัติ!  
    print(animals); // Output: {Cat, Dog, Bird}  
  
    // 2. การสร้าง Set ว่างๆ (พิมพ์แบบกำหนด Type)  
    Set<int> uniqueNumbers = {};  
  
    // 3. การเพิ่มข้อมูล (.add และ .addAll)  
    uniqueNumbers.add(10);  
    uniqueNumbers.add(20);  
    uniqueNumbers.add(10); // ลองเพิ่ม 10 ซ้ำอีกครั้ง  
    uniqueNumbers.addAll({'30', 40, 50});  
  
    print(uniqueNumbers); // Output: {10, 20, 30, 40, 50} (10 ไม่ซ้ำ)  
  
    // 4. การตรวจสอบว่ามีข้อมูลอยู่ไหม (ทำงานได้เร็วมากใน Set)  
    if (uniqueNumbers.contains(30)) {  
        print('มีเลข 30 อยู่ในเซตนี้!');  
    }  
  
    // 5. การลบข้อมูล  
    uniqueNumbers.remove(20);  
    print(uniqueNumbers); // Output: {10, 30, 40, 50}  
}
```

การประกาศตัวแปรพร้อมกำหนดค่าเริ่มต้นใช้เครื่องหมาย { }

การประกาศตัวแปรประเภท set โดยใช้เครื่องหมาย { }

การกำหนดค่าให้กับตัวแปรประเภท Set โดยใช้ add, addAll

ตัวแปร Set (cont.)

- การทำ Intersections (หาตัวซ้ำ) หรือ Unions (รวมกัน) ของตัวแปรประเภท Set

```
void main() {  
    var tagTeamA = {'Tom', 'John', 'Alice'};  
    var tagTeamB = {'Alice', 'Bob', 'Chris'};  
  
    // 1. Intersection (หาคนที่ชื่ออยู่ในทั้งสองกลุ่ม)  
    var bothTeams = tagTeamA.intersection(tagTeamB);  
    print(bothTeams); // Output: {Alice}  
  
    // 2. Union (รวมรายชื่อของทั้งสองกลุ่มเข้าด้วยกัน โดยไม่ให้ชื่อซ้ำ)  
    var allMembers = tagTeamA.union(tagTeamB);  
    print(allMembers); // Output: {Tom, John, Alice, Bob, Chris}  
  
    // 3. Difference (หาคนที่ชื่ออยู่ใน Team A แต่ไม่อยู่ใน Team B)  
    var onlyTeamA = tagTeamA.difference(tagTeamB);  
    print(onlyTeamA); // Output: {Tom, John}  
}
```

Operators in Dart

TABLE 3.1: Arithmetic operators

OPERATOR	DESCRIPTION	SAMPLE CODE
+	Add	<code>7 + 3 = 10</code>
-	Subtract	<code>7 - 3 = 4</code>
*	Multiply	<code>7 * 3 = 21</code>
/	Divide	<code>7 / 3 = 2.33</code>

TABLE 3.5: Logical operators

OPERATOR	DESCRIPTION	SAMPLE CODE
!	! is a logical 'not'. Returns the opposite value of the variable/expression.	<code>if (!(7 > 3)) = false</code>
&&	&& is a logical 'and'. Returns true if the values of the variable/expression are all true.	<code>if ((7 > 3) && (3 < 7)) = true</code> <code>if ((7 > 3) && (3 > 7)) = false</code>
!!	!! is a logical 'or'. Returns true if at least one value of the variable/expression is true.	<code>if ((7 > 3) (3 > 7)) = true</code> <code>if ((7 < 3) (3 > 7)) = false</code>

Operators in Dart (cont.)

TABLE 3.2: Equality and relational operators

OPERATOR	DESCRIPTION	SAMPLE CODE
<code>==</code>	Equal	<code>7 == 3 = false</code>
<code>!=</code>	Not equal	<code>7 != 3 = true</code>
<code>></code>	Greater than	<code>7 > 3 = true</code>
<code><</code>	Less than	<code>7 < 3 = false</code>
<code>>=</code>	Greater than or equal to	<code>7 >= 3 = true</code> <code>4 >= 4 true</code>
<code><=</code>	Less than or equal to	<code>7<= 3 = false</code> <code>4 <= 4 = true</code>

TABLE 3.4: Assignment operators

OPERATOR	DESCRIPTION	SAMPLE CODE
<code>=</code>	Assigns value	<code>7 = 3 = 3</code>
<code>??=</code>	Assigns value only if variable being assigned to has a value of null	<code>Null ??= 3 = 3</code> <code>7 ??= 3 = 7</code>
<code>+=</code>	Adds to current value	<code>7 += 3 = 10</code>
<code>-=</code>	Subtracts from current value	<code>7 -= 3 = 4</code>
<code>*=</code>	Multiplies from current value	<code>7 *= 3 = 21</code>
<code>/=</code>	Divides from current value	<code>7 /= 3 = 2.33</code>

Operators in Dart – Null aware (cont.)

- Operator ?? - ถ้าด้านซ้ายเป็น null จะใช้ค่าด้านขวา หากด้านซ้ายไม่เป็น null จะใช้ด้านซ้าย เช่น

```
1 void main() {  
2     print(0 ?? 1); // show 0  
3     print(null ?? 1); // show 1  
4     print(null ?? null ?? 2); // show 2  
5 }
```

- สามารถนำไปประยุกต์กับการ Assign ค่าให้กับตัวแปรได้ เช่น หากค่าที่ต้องการเป็น Null ก็ให้อีกค่าหนึ่งทดแทน เป็นต้น

```
var name = getFromDatabase() ?? 'Not available';
```


Loops

- การวนซ้ำโดยใช้ while และ for

```
Run | Debug
1 void main() {
2     List<double> score = [50, 60, 65, 75, 89, 45, 56, 40, 72, 80];
3     var i = 0;
4
5     while (i < score.length) {
6         print(score[i]);
7         i++;
8     }
9 }
```

```
Run | Debug
1 void main() {
2     List<double> score = [50, 60, 65, 75, 89, 45, 56, 40, 72, 80];
3     var i;
4
5     for (i = 0; i < score.length; i++) {
6         print(score[i]);
7     }
8 }
9
```


Loops

- การวนซ้ำโดยใช้ foreach และ for in

```
Run | Debug
1 void main() {
2     List<double> score = [50, 60, 65, 75, 89, 45, 56, 40, 72, 80];
3
4     score.forEach((element) {
5         print(element);
6     });
7 }
8
```

```
Run | Debug
1 void main() {
2     List<double> score = [50, 60, 65, 75, 89, 45, 56, 40, 72, 80];
3
4     for (double sc in score) {
5         print(sc);
6     }
7 }
8
```

Condition

- if-else

ในส่วนของ else จะมีหรือไม่มีก็ได้ แต่หากจำเป็นต้องมีจะต้องใช้คู่กับ if

```
if(boolean_expression){  
    // statement(s) will execute if the Boolean expression is true.  
} else {  
    // statement(s) will execute if the Boolean expression is false.  
}
```

```
1 void main() {  
2     var num = 12;  
3     if (num % 2==0) {  
4         print("Even number");  
5     } else {  
6         print("Odd number");  
7     }  
8 }
```

Function

- เป็นการรวบรวมคำสั่งไว้เป็นกลุ่มย่อย เพื่อสามารถนำมาเรียกใช้ซ้ำได้ในภายหลัง

```
<return type> function_name ([parameters]) {  
    // body of function  
    // ...  
    return <value>  
}
```

หาก return type เป็นคำว่า **void** ฟังก์ชันนั้น ๆ ไม่ต้องมีคำสั่ง **return**

```
1  double add2Number(double n1, double n2) {  
2  |   return n1 + n2;  
3  |   }  
4  
   Run | Debug  
5  void main() {  
6  |   double r1 = add2Number(10, 20);  
7  |   print('result = ${r1}');  
8  |   }
```

```
1  double add2Number(double n1, double n2) {  
2  |   return n1 + n2;  
3  |   }  
4  
5  double minNumber(double n1, double n2) {  
6  |   if (n1 < n2)  
7  |   |   return n1;  
8  |   else  
9  |   |   return n2;  
10 |   }  
11
```

Run | Debug

```
12 void main() {  
13 |   double r1 = add2Number(10, 20);  
14 |   print('result = ${r1}');  
15  
16 |   double r2 = minNumber(100, 15);  
17 |   print('min = ${r2}');  
18 |   }  
19
```

Arrow syntax (=>), และ Named Parameters

- **Arrow Function** (=>) ในภาษา Dart เครื่องหมาย => (เรียกว่า Arrow syntax) เป็นทางลัด (Shorthand) ที่ใช้สำหรับเขียนฟังก์ชันที่มีโค้ดภายในเพียงบรรทัดเดียว มีความหมายคือการแทนที่วงเล็บปีกกา { } และคำสั่ง return เพื่อให้โค้ดดูสะอาด สั้น และอ่านง่ายขึ้น (*หรือเปล่า?*)

```
// แบบปกติ (Normal Function)
double calculateAreaNormal(double width, double height) {
    return width * height;
}

// แบบ Arrow Function (สั้นลงมาก และส่งค่ากลับอัตโนมัติ)
double calculateAreaArrow(double width, double height) => width * height;

void main() {
    print(calculateAreaNormal(5, 10)); // Output: 50.0
    print(calculateAreaArrow(5, 10));  // Output: 50.0
}
```

Arrow syntax ($\Rightarrow$), และ Named Parameters

- **Named Parameters** (พารามิเตอร์แบบระบุชื่อ) ปกติฟังก์ชันทั่วไป (เรียกว่า Positional Parameters) เวลาเรียกใช้งาน จะต้องส่งค่าเข้าไปให้ถูกลำดับ เช่น ฟังก์ชันต้องการ (ชื่อ, อายุ) เราก็ต้องส่งชื่อก่อนแล้วตามด้วยอายุห้ามสลับกัน แต่ Named Parameters จะลบข้อจำกัดนั้นทิ้งไป โดยเวลาเรียกใช้ เราสามารถส่งค่าตามชื่อตัวแปรได้เลย โดยสามารถสลับลำดับกันได้
- การประกาศตัวแปรที่ใช้เป็นพารามิเตอร์ของฟังก์ชัน จะใช้เครื่องหมาย { } ครอบพารามิเตอร์ทั้งหมดไว้

การใช้งาน Widget ที่อยู่ใน Flutter SDK จะใช้การส่งค่าแบบ Named Parameters

ตัวอย่าง Named Parameters

```
// ประกาศฟังก์ชันโดยใช้ ๕ ๕ คอปรพพารามิเตอร์
// ใช้ 'required' ถ้านั้นจำเป็นต้องส่งมา และระบุค่า Default ได้ถ้าไม่ยอกให้เป็น Null
void createUserProfile({
    required String name,
    required int age,
    String country = 'Thailand', // ถ้าไม่ส่งมา จะใช้ค่านี้อัตโนมัติ
}) {
    print('Name: $name, Age: $age, Country: $country');
}

void main() {
    // เวลาเรียกใช้งาน ต้องใส่ "ชื่อพารามิเตอร์:" กำกับไว้เสมอ
    // และข้อดีคือ สลับเอา age ขึ้นก่อน name ก็ได้!
    createUserProfile(
        age: 25,
        name: 'John Doe',
    );
    // Output: Name: John Doe, Age: 25, Country: Thailand (ใช้ค่า Default)

    createUserProfile(
        name: 'Lisa',
        age: 22,
        country: 'South Korea', // ส่งค่าใหม่ไปทับค่า Default
    );
    // Output: Name: Lisa, Age: 22, Country: South Korea
}
```

คำว่า **required** ที่นำหน้าชื่อตัวแปร จะหมายถึงเมื่อมีการเรียกใช้ฟังก์ชันนี้ ผู้เรียกจะต้องส่งค่าของตัวแปรที่มีการ required มาด้วยเสมอขาดไปไม่ได้

ข้อแตกต่าง: พารามิเตอร์แบบปกติ VS แบบระบุชื่อ (Named Parameters)

```
// ประกาศฟังก์ชันแบบปกติ (อิงตามลำดับที่ส่ง)
void createButton(String text, String color, double width, double height) {
    print("สร้างปุ่ม: $text, สี: $color, กว้าง: $width, สูง: $height");
}

void main() {
    // เวลาเรียกใช้: ต้องจำให้ได้ว่าตัวไหนมาก่อนมาหลัง! ห้ามสลับลำดับเด็ดขาด
    createButton("ตกลง", "สีน้ำเงิน", 150.0, 50.0);

    // โค้ดอ่านยาก: คนมาอ่านทีหลังจะไม่รู้เลยว่า 150.0 กับ 50.0 ตัวไหนคือกว้างหรือสูง
    createButton("ยกเลิก", "สีแดง", 100.0, 50.0);
}
```

ข้อแตกต่าง: พารามิเตอร์แบบปกติ VS แบบระบุชื่อ (Named Parameters)

```
// ประกาศฟังก์ชันแบบ Named Parameters (ใส่ปีกกาครอบ)
void createButton({
    required String text,    // ต้องส่งมาเสมอ ขาดไม่ได้
    required String color,   // ต้องส่งมาเสมอ ขาดไม่ได้
    double width = 120.0,    // ไม่ส่งมาได้ เพราะมีค่าเริ่มต้น (Default Value)
    double height = 45.0,    // ไม่ส่งมาได้ เพราะมีค่าเริ่มต้น (Default Value)
}) {
    print("สร้างปุ่ม: $text, สี: $color, กว้าง: $width, สูง: $height");
}

void main() {
    // เวลาเรียกใช้: ใส่ "ชื่อพารามิเตอร์:" กำกับ และสลับลำดับอย่างไรก็ได้!
    createButton( color: "สีน้ำเงิน", text: "ตกลง", width: 200.0, height: 60.0 );

    // สามารถระบุเฉพาะค่าที่จำเป็น ส่วนที่เหลือ width กับ height จะใช้ค่า Default อัตโนมัติ
    createButton( text: "ส่งข้อมูล", color: "สีเขียว" );
}
```


Non-Nullable/Null safety

- การป้องกันไม่ให้เกิดปัญหาเกี่ยวกับค่าตัวแปรที่เป็นค่า **null** ภาษา Dart จึงไม่ยอมให้เราประกาศตัวแปรโดยไม่กำหนดค่าเริ่มต้นให้กับตัวแปรนั้น
- แต่หากประสงค์จะประกาศโดยไม่กำหนดค่าเริ่มต้นก็สามารถทำได้เช่นกัน แต่จะต้องใส่ข้อมูลเพิ่มเติมเพื่อบอก Compiler ให้ทราบว่าต้องการแบบนั้น

Non-nullable/null safety (cont.)

- ตัวอย่าง

```
Run | Debug
1 void main() {
2   String? name; // name มีโอกาสเป็น null หรือไม่ก็ได้
3   String surname; // surname จะเป็น null ไม่ได้
4
5   print('${name} ${surname}');
6 }
7
```

❌ hello.dart 1 of 1 problem

The non-nullable local variable 'surname' must be assigned before it can be used.
Try giving it an initializer expression, or ensure that it's assigned on every execution path. dart([not_assigned_potentially_](#)

```
Run | Debug
1 void main() {
2   String? name; // name มีโอกาสเป็น null หรือไม่ก็ได้
3   String surname = ''; // surname จะเป็น null ไม่ได้ ตั้งค่าเริ่มต้นเป็นค่าว่าง
4
5   print('${name} ${surname}');
6 }
7
```

Non-nullable/null safety (cont.)

- การรับส่งค่าจากตัวแปรที่ยอมให้เป็น null ได้ ไปให้กับตัวแปรที่ไม่ยอมให้เป็น null ...จะเกิดปัญหา
- ดังตัวอย่าง ตัวแปร `String? name` กับตัวแปร `String n`

```
1 void showMessage(String n, String s) {
2   print('$n $s');
3 }
4
Run | Debug
5 void main() {
6   String? name; // name มีโอกาสเป็น null หรือไม่ก็ได้
7   String surname = ''; // surname จะเป็น null ไม่ได้ ตั้งค่าเริ่มต้นเป็นค่าว่าง
8
9   showMessage(name, surname);
10 }
```

hello.dart 1 of 1 problem

The argument type 'String?' can't be assigned to the parameter type 'String'. dart([argument_type_not_assignable](#))

แนวทางแก้ไข (1) 9 `showMessage(name.toString(), surname);`

แนวทางแก้ไข (2) - ทำให้ตัวแปรรับและส่งอยู่ในลักษณะเดียวกัน

Spread Operator (...)

- Spread Operator (...) ในภาษา Dart คือเครื่องหมายที่ใช้สำหรับ "การกระจาย" สมาชิกที่อยู่ใน Collection (เช่น List, Set หรือ Map) ออกมาทีละตัว เพื่อนำไปใส่ในอีก Collection หนึ่ง

```
void main() {  
  var fruits = ['Apple', 'Banana'];  
  
  // ใช้ ... เพื่อดึง Apple และ Banana ออกมาวางใน List ใหม่  
  var allFruits = ['Mango', ...fruits, 'Orange'];  
  
  print(allFruits);  
  // ผลลัพธ์: [Mango, Apple, Banana, Orange]  
}
```

Spread Operator (...)

- ตัวอย่างเมื่อใช้งานกับ map

```
void main() {  
    var baseConfig = {'theme': 'dark', 'notifications': true};  
  
    var userConfig = {  
        'username': 'gemini_user',  
        ...baseConfig, // กระจายค่าจาก baseConfig เข้ามา  
    };  
  
    print(userConfig);  
    // ผลลัพธ์: {username: gemini_user, theme: dark, notifications: true}  
}
```

Null-aware Spread Operator (...?)

- ในกรณีที่ List ต้นทาง อาจเป็นค่าว่าง (null) หากเราใช้ ... ปกติ โปรแกรมจะแสดง Error ดังนั้น Dart จึงมีเครื่องหมาย ...? มาให้เพื่อความปลอดภัย

```
void main() {  
    List<int>? additionalNumbers; // ตัวแปรนี้เป็น null  
  
    var numbers = [1, 2, 3, ...?additionalNumbers];  
  
    print(numbers);  
    // ผลลัพธ์: [1, 2, 3] (ไม่ Error แม้ตัวแปรจะเป็น null)  
}
```

- ... = เอาของข้างในออกมาวางเรียงกันในที่ใหม่
- ...? = เหมือนด้านบน แต่ถ้าของข้างในไม่มี (null) ก็ให้ข้ามไปโดยไม่ต้องแสดง Error

OOP in Dart

- Classes / Abstract Classes
- Instance of Class
- Constructor
- Inheritance
- Overriding
- Access Modifier (public/private)

โครงสร้างของคลาส

- Use the **class** keyword to declare a **class** in Dart. A class definition starts with the keyword **class** followed by the **class name**; and the class body enclosed by a pair of curly braces.

```
class class_name {  
    <fields>  
    <getters/setters>  
    <constructors>  
    <functions>  
}
```

- เวลาบันทึก ชื่อไฟล์ไม่จำเป็นต้องตั้งชื่อเดียวกับชื่อคลาส
- ภายใน 1 ไฟล์สามารถมีได้มากกว่า 1 คลาส

- **Fields** – any variable declared in a class.
- **Setters and Getters** – Allows the program to initialize and retrieve the values of the fields of a class.
- **Constructors** – responsible for allocating memory for the objects of the class.
- **Functions/Methods** – Functions represent actions an object can take.

ตัวอย่างคลาส

```
1  class Person {  
2      String name;  
3      int age;  
4  
5      // Constructor  
6      Person(this.name, this.age);  
7  
8      // Method to greet  
9      // This method prints a greeting message  
10     void greet() {  
11         print("Hello, my name is $name and I am $age years old.");  
12     }  
13 }  
14  
15
```

Run | Debug

```
16 void main() {  
17     // Create an instance of Person  
18     Person p = Person("Somsak", 30);  
19  
20     // Call the greet method  
21     p.greet();  
22 }
```

- การสร้าง object ของ Person จะมีคำว่า new นำหน้าชื่อคลาสหรือไม่ก็ได้
- ในตัวอย่างนี้ไม่มีคำว่า new

แนวคิดเกี่ยวกับ Constructor รูปแบบต่างๆ ของ Dart

- เนื่องจากใน Flutter "ทุกอย่างคือ Widget" และ Widget เกือบทั้งหมดถูกสร้างผ่าน Constructor
- **Initializing Formal Parameters (Shorthand Constructor):** รูปแบบการเขียน Constructor ย่อที่นิยมที่สุดใน Dart เช่น `Person(this.name, this.age);`; แทนการเขียนกำหนดค่าใน Body `{ }` แบบดั้งเดิม
- **Named Constructors:** การสร้าง Constructor หลายแบบในคลาสเดียว เช่น `Person.fromJson(Map data)` ซึ่งจะพบบ่อยมากเมื่อแปลงข้อมูล JSON จาก Web API มาใช้ในแอปพลิเคชัน

แนวคิดเกี่ยวกับ Constructor รูปแบบต่างๆ ของ Dart

```
class Product {  
  final String name;  
  final double price;  
  final int stock;  
  // 1. Default Constructor: สำหรับใช้สร้าง Object ทั่วไปในแอป  
  Product({required this.name, required this.price, required this.stock});  
  
  // 2. Named Constructor: จาก Map/JSON (ใช้บ่อยที่สุดเมื่อดึงข้อมูลจาก API)  
  // รับข้อมูลแบบ Map เข้ามา แล้วจับคู่แยกใส่แต่ละ Property  
  // ใช้สไลซ์ : (Initializer List)  
  Product.fromJson(Map<String, dynamic> json)  
    : name = json['product_name'] ?? 'ไม่มีชื่อสินค้า',  
      price = (json['price'] ?? 0).toDouble(),  
      stock = json['qty'] ?? 0;  
  
  // 3. Named Constructor: จากการคำนวณหรือแปลงข้อมูลรูปแบบเฉพาะ  
  // ใช้ factory เข้ามาช่วยจัดการข้อมูลดิบก่อนส่งต่อ  
  // สมมติว่ารับข้อมูลดิบเป็นข้อความบรรทัดเดียวคั่นด้วยเครื่องหมาย Comma (CSV format)  
  factory Product.fromCsvLine(String csvLine) {  
    // แยกข้อความด้วยเครื่องหมาย ',' เช่น "iPhone,32000,10"  
    List<String> parts = csvLine.split(',');  
    // แปลงข้อมูลและกำหนดค่าให้ Property หลังจากการประมวลผลข้อมูลเสร็จ  
    String iname = parts[0];  
    double iprice = double.parse(parts[1]);  
    int istock = int.parse(parts[2]);  
  
    return Product(name: iname, price: iprice, stock: istock);  
  }  
}
```

แนวคิดเกี่ยวกับ Constructor รูปแบบต่างๆ ของ Dart (cont.)

```
void main() {  
  // --- ทดสอบแบบที่ 1: สร้าง Object ปกติ ---  
  var normalProduct = Product(name: 'คีย์บอร์ด', price: 1500, stock: 5);  
  print('1. แบบปกติ: ${normalProduct.name} ราคา ${normalProduct.price} บาท');  
  
  // --- ทดสอบแบบที่ 2: แปลงข้อมูลจาก JSON (Map) ที่จำลองมาจาก API ---  
  Map<String, dynamic> apiResponse = {  
    'product_name': 'เมาส์ไร้สาย',  
    'price': 890.50,  
    'qty': 20  
  };  
  
  // เรียกใช้ .fromJson() เพื่อแปลงโครงสร้างข้อมูล  
  var apiProduct = Product.fromJson(apiResponse);  
  print('2. จาก JSON API: ${apiProduct.name} (คลังเหลือ: ${apiProduct.stock})');  
  
  // --- ทดสอบแบบที่ 3: แปลงข้อมูลจากบรรทัดข้อความ (CSV Line) ---  
  String csvData = "หูฟังบลูทูธ,2500,12";  
  
  // เรียกใช้ .fromCsvLine() เพื่อตัดคำแล้วแปลงเป็น Object  
  var csvProduct = Product.fromCsvLine(csvData);  
  print('3. จาก CSV string: ${csvProduct.name} ราคา ${csvProduct.price} บาท');  
}
```

แนวคิดเกี่ยวกับ Constructor รูปแบบต่างๆ ของ Dart (cont.)

- ข้อสรุปการใช้รูปแบบต่าง ๆ ในการแปลงข้อมูล ระหว่าง `Initializer List(:)` และ `factory`
 - ถ้าข้อมูลที่ได้รับมาสามารถจับคู่ (Mapping) ได้ทันที เช่น โครงสร้าง JSON/Map -> แนะนำให้ใช้ `Initializer List (:)`
 - ถ้าข้อมูลที่ได้รับมาต้องมีการจัดการก่อน เช่น การตัดคำ การแยกข้อความ หรือคำนวณสูตรต่างๆ -> แนะนำให้ใช้ `Factory Constructor (factory)` ร่วมกับตัวแปร `final`

ตัวอย่างการกำหนดการเข้าถึง (Access Modifier)

- ตัวแปร หรือ *method* ใดของ class ที่มีเครื่องหมาย `_` นำหน้า เช่น `_name`, `_age` จะถือว่าเป็น `private` ไม่สามารถเข้าถึงได้โดยตรงจากภายนอก

หากต้องการแก้ไขชื่อและอายุใน object เดิม ก็สามารถเรียกใช้ Setter method ที่สร้างไว้ได้ เช่น ตัวอย่างด้านล่างนี้

```
35 void main() {  
36     // Create an instance of Person  
37     Person p = Person("Somsak", 30);  
38     // Call greet method of class Person  
39     p.greet();  
40     // Call my greet method  
41     myGreet(p.name, p.age);  
42     p.name = "Somchai";  
43     p.age = 25;  
44     // Call greet method of class Person again  
45     p.greet();  
46 }
```

```
1 class Person {  
2     String _name;  
3     int _age;  
4  
5     // Constructor  
6     Person(this._name, this._age);  
7  
8     void set name(String name) {  
9         _name = name;  
10    }  
11  
12    void set age(int age) {  
13        _age = age;  
14    }  
15  
16    String get name {  
17        return _name;  
18    }  
19  
20    int get age {  
21        return _age;  
22    }  
23  
24    // Method to greet  
25    // This method prints a greeting message  
26    void greet() {  
27        print("Hello, my name is $name and I am $age years old.");  
28    }  
29  
30  
31    void myGreet(String myname, int myage) {  
32        print("Sawasdee, my name is $myname and I am $myage years old.");  
33    }  
34  
35    Run | Debug  
36    void main() {  
37        // Create an instance of Person  
38        Person p = Person("Somsak", 30);  
39        // Call greet method of class Person  
40        p.greet();  
41        // Call my greet method  
42        myGreet(p.name, p.age);  
43    }  
44 }
```

การสืบทอด – Inheritance/Overriding

```
1 class Person {
2     String _name;
3     int _age;
4
5     // Constructor
6     Person(this._name, this._age);
7
8     void set name(String name) {
9         _name = name;
10    }
11
12    void set age(int age) {
13        _age = age;
14    }
15
16    String get name {
17        return _name;
18    }
19
20    int get age {
21        return _age;
22    }
23
24    // Method to greet
25    // This method prints a greeting message
26    void greet() {
27        print("Hello, my name is $name and I am $age years old.");
28    }
29 }
```

```
31 class Student extends Person {
32     String _studentID;
33
34     // Constructor
35     Student(String name, int age, this._studentID) : super(name, age);
36
37     void set studentID(String studentID) {
38         _studentID = studentID;
39     }
40
41     String get studentID {
42         return _studentID;
43     }
44 }
45
46 Run | Debug
47 void main() {
48     // Create a Person object
49     Person person = Person("Somsak", 30);
50     person.greet(); // Output: Hello, my name is Somsak and I am 30 years old.
51
52     // Create a Student object
53     Student student = Student("Somchai", 20, "S12345");
54     student.greet(); // Output: Hello, my name is Somchai and I am 20 years old.
55 }
```

ใช้คำว่า extends ในการสืบทอด

เรียก method ชื่อ greet() จากแม่

การสืบทอด – Inheritance/Overriding

```
1 class Person {
2     String _name;
3     int _age;
4
5     // Constructor
6     Person(this._name, this._age);
7
8     void set name(String name) {
9         _name = name;
10    }
11
12    void set age(int age) {
13        _age = age;
14    }
15
16    String get name {
17        return _name;
18    }
19
20    int get age {
21        return _age;
22    }
23
24    // Method to greet
25    // This method prints a greeting message
26    void greet() {
27        print("Hello, my name is $name and I am $age years old.");
28    }
29 }
```

```
31 class Student extends Person {
32     String _studentID;
33
34     // Constructor
35     Student(String name, int age, this._studentID) : super(name, age);
36
37     void set studentID(String studentID) {
38         _studentID = studentID;
39     }
40
41     String get studentID {
42         return _studentID;
43     }
44
45     // Method to greet
46     // This method prints a greeting message
47     @override
48     void greet() {
49         print("Hello, my name is $name and I am $age years old. My student ID is $studentID.");
50     }
51 }
52
53 Run | Debug
54 void main() {
55     // Create a Person object
56     Person person = Person("Somsak", 30);
57     person.greet(); // Output: Hello, my name is Somsak and I am 30 years old.
58
59     // Create a Student object
60     Student student = Student("Somchai", 20, "S12345");
61     student.greet();
62     // Output: Hello, my name is Somchai and I am 20 years old. My student ID is S12345.
63 }
```

สร้าง greet() ของตนเอง โดยใช้ชื่อซ้ำกับแม่

เรียก method ชื่อ greet() ซึ่งผลลัพธ์จะแตกต่างจากเดิม
เนื่องจากการเรียกใช้ greet() ของตนเองที่สร้างขึ้นใหม่

Abstraction

- เป็น Class ที่ยังมีบางส่วนไม่สมบูรณ์ จำเป็นต้องให้ผู้สืบทอดนำไปขยายความต่อให้สมบูรณ์

```
1 abstract class Person {
2     String _name;
3     int _age;
4
5     // Constructor
6     Person(this._name, this._age);
7
8     void set name(String name) {
9         | _name = name;
10    }
11
12    void set age(int age) {
13        | _age = age;
14    }
15
16    String get name {
17        | return _name;
18    }
19
20    int get age {
21        | return _age;
22    }
23
24    // Abstract Method to greet
25    void greet();
26 }
```

```
28 class ThaiPeople extends Person {
29
30     // Constructor
31     ThaiPeople(String name, int age) : super(name, age);
32
33     // Override the greet method
34     // This method prints a greeting message
35     @override
36     void greet() {
37         | print("Sawasdee, my name is $name and I am $age years old");
38     }
39 }
40
41 class ChinesePeople extends Person {
42
43     // Constructor
44     ChinesePeople(String name, int age) : super(name, age);
45
46     // Override the greet method
47     // This method prints a greeting message
48     @override
49     void greet() {
50         | print("Ni Hao, my name is $name and I am $age years old");
51     }
52 }
53
54 void main() {
55     // Create a Person object
56     Person person = ThaiPeople("Somsak", 30);
57     person.greet(); // Output: Sawasdee, my name is Somsak and I am 30 years old.
58
59     // Create a ChinesePeople object
60     Person person2 = ChinesePeople("Li", 25);
61     person2.greet(); // Output: Ni Hao, my name is Li and I am 25 years old.
62 }
```

การเรียกใช้ Class จากไฟล์อื่น

- ใช้คำสั่ง import ตามด้วยชื่อไฟล์ที่บันทึกคลาสนั้น ๆ ไว้ เช่น หากคลาส Person ถูกบันทึกไว้ในไฟล์ชื่อ person.dart และ หากต้องการเรียกใช้ จะสามารถใช้คำสั่ง import เพื่อดึงเข้ามาได้ ดังตัวอย่าง

```
1  import 'person.dart';
2
3  Run | Debug
4  void main() {
5      // Create a Person object
6      Person person = ThaiPeople("Somsak", 30);
7      person.greet(); // Output: Sawasdee, my name is Somsak and I am 30 years old.
8
9      // Create a ChinesePeople object
10     Person person2 = ChinesePeople("Li", 25);
11     person2.greet(); // Output: Ni Hao, my name is Li and I am 25 years old.
12 }
```

การทำงานแบบ Asynchronous Programming

- แอปพลิเคชันมือถือนี้อาจต้องมีการดึงข้อมูลจากอินเทอร์เน็ต อ่านไฟล์ หรือเปิดฐานข้อมูล ซึ่งงานเหล่านี้มักต้องทำแบบเบื้องหลัง (Background) เพื่อไม่ให้หน้าจอแอปฯ ค้าง
- Keyword ที่สำคัญในภาษา Dart ที่เกี่ยวข้องมีอยู่ 3 คำคือ: **Future**, **async**, และ **await**
 - **Future**: เปรียบเสมือน "หมายเลขคิว" หรือใบเสร็จที่ได้มาตอนสั่งอาหาร ซึ่งไม่ใช่อาหารจริงๆ แต่เป็นคำสัญญาว่า "เราจะได้รับอาหารในอนาคตนะ" (เปรียบได้กับข้อมูลที่ยังมาไม่ถึง แต่จะได้มาในอนาคต)
 - **async**: คำสั่งที่ใช้วางไว้หลังชื่อฟังก์ชัน เพื่อบอกให้ Dart รู้ว่า "ฟังก์ชันนี้มีงานที่ต้องรอนะ"
 - **await**: คำสั่งที่วางไว้หน้าคำสั่ง ใช้บอกให้ Dart "รอตตรงนี้แป๊บหนึ่งนะ" จนกว่างานจะทำเสร็จแล้วค่อยไปยังบรรทัดถัดไป (หากมีการใช้ await ฟังก์ชันนั้น ๆ จะต้องระบุคำว่า async ไว้ด้วยเท่านั้น)

การทำงานแบบ Asynchronous Programming (cont.)

```
// ฟังก์ชันจำลองการต้มน้ำ ซึ่งต้องใช้เวลา 3 วินาที
// เราจะคืนค่าเป็น Future<String> เพราะตอนสั่ง ต้มน้ำยังไม่เสร็จทันที ต้องรอในอนาคต
Future<String> boilWater() async {
  print('1. เริ่มต้มน้ำชาวด...');

  // จำลองการรอน้ำเดือด 3 วินาที (ไม่ทำให้โปรแกรมค้าง)
  await Future.delayed(Duration(seconds: 3));

  return 'น้ำเดือดจัด!'; // คืนค่ากลับไปเมื่อครบ 3 วินาที
}

void main() async {
  print('--- เริ่มกระบวนการทำอาหาร ---');

  // สั่งต้มน้ำ (มี await แปลว่าเราจะรอให้จบกระบวนการต้มน้ำตรงนี้ก่อน)
  String waterStatus = await boilWater();
  print('2. ผลลัพธ์: $waterStatus');

  // งานถัดไปทำหลังจากน้ำเดือดแล้ว
  print('3. เทน้ำร้อนใส่ถ้วยมามา แล้วทานได้เลย!');
  print('--- จบกระบวนการ ---');
}
```

ผลลัพธ์

```
--- เริ่มกระบวนการทำอาหาร ---
1. เริ่มต้มน้ำชาวด...
(น้ำจจะนิ่งไป 3 วินาทีเบื้องหลัง แต่แอปไม่ได้ค้าง)
2. ผลลัพธ์: น้ำเดือดจัด!
3. เทน้ำร้อนใส่ถ้วยมามา แล้วทานได้เลย!
--- จบกระบวนการ ---
```

Q/A